Date 27/07/2024

Expt. No. 1. Page No. 1.

P.S.1 Lex Program to count the number of vowels & consonants in code.

Compiler Design :- Compiler is a software / program that converts a program written in a high level languages such as C, C++, or python into low-level / machine language.

Main Objectives :-

Lexical Analysis
Parser / Syntax Analysis (parsing)
Intermediate code generation
Code optimization
Code generation.

-> Understand semantic analysis & error handling.

LEX

- Lexical Analysis is the first phase of a compiler.
- The input program is given to the compiler.
- The program is devided into tokens, which are used for further lexical analysis.
- Tokens are the smallest meaningful units of a program.
- Tokens can include identifiers, keywords, operators, constants & symbols.
- Words & characters such as a, 0, +, etc. are identified during lexical ~~symbols~~. Analysis.
- The main purpose of lexical analysis is to convert the input character stream into a stream of tokens.
- It also helps identify lexical error in the source code.

Teacher's Signature _____

The output of the lexical analyzer is passed to the syntax analyzer / parser.

Ex:

for the statement

int a = 10 ;

int ⇒ keyword

a ⇒ identifier

= ⇒ Operator

10 ⇒ Constant (literal)

; ⇒ Delimiter.

This experiment demonstrates the basic working of LEX pattern & actions.

Algorithm :-

1. Start the program
2. Initialize vowel_count & consonant_count to 0
3. Define a LEX pattern for vowels.
4. Define a LEX pattern for consonants.
5. Read the input C file using LEX.
6. If a vowel is found, increment vowel_count.
7. If a consonant found, increment consonant_count.
8. Ignore all other characters.
9. Print the total number of vowels & consonants.
10. Stop.

Date

Expt. No. _____ Page No. 3.

Lex Code in C

Code :-

```
%{
#include <stdio.h>
int vowel_count = 0;
int consonant_count = 0;
%}
%%%
[aeiouAEIOU] { vowel_count ++; }

[b-df-hj-np-tv-zB-DF-HJ-NP-TV-Z]
                { Consonant_count ++; }
.|\n        { /* ignore other characters */ }

%%%
int main()
  {
    yylex();
      printf("Number of vowels = %d\n",
                vowel_count);
printf("Number of consonants = %d\n", consonant_count);
 return 0;
}
int yywrap()
{
 return 1;
}
```

Result :- The LEX program was successfully executed & the total number of vowels & consonants present in the given input file are counted.

Teacher's Signature _____

Viva Question :-

Q1 What is LEX & what is its role in CD?

Ans LEX is a tool used to generate a lexical analyzer automatically. It identifies patterns in the source code & converts the input into the token such that keywords, identifiers, operators & constants.

Q2 How do you differentiate between vowel & consonants in your LEX program?

Ans Vowels are identified using the pattern [aeiouAEIOU]. Consonants are identified by matching the remaining alphabatic characters, while numbers, spaces & symbols are ignored.

Q3 How does LEX handles overlapping patterns?

Ans When multiple patterns match the input, LEX selects the longest possible match. If two patterns match the same number of characters, the pattern defined first in the LEX program gets priority.

Q4 What is the purpose of yytext function/variable?

Ans yytext stores the text currently matched by the lexical analyzer. It can be used inside an action to access, display, or process the matched sequence of characters.

Teacher's Signature

Q.5 How do you handle uppercase & lowercase letters in your pattern matching?

Ans We include both uppercase & lowercase letters in the LEX pattern. For example, [aeiouAEIOU] matched vowels in both cases, such as 'A', 'e', 'a' & 'E'